

データマイニング特論

第5回

分類の基礎 — K近傍法と決定木

本科目シラバスの確認 3（授業計画）

#	日程	内容
第1回	4/16（木）	ガイダンス、イントロダクション+Python環境構築
第2回	4/23（木）	pandasによるデータ操作の基礎
第3回	4/30（木）	探索的データ分析（EDA）と可視化（オンデマンド）
第4回	5/7（木）	データの前処理と特徴量エンジニアリング
第5回	5/14（木）	分類の基礎 — k近傍法と決定木
第6回	5/21（木）	アンサンブル学習
第7回	5/28（木）	線形分類モデルと回帰
第8回	6/4（木）	モデル評価・選択と不均衡データ
第9回	6/11（木）	クラスタリング手法
第10回	6/18（木）	アソシエーション分析と異常検知
第11回	6/25（木）	テキストマイニング
第12回	7/2（木）	時系列データマイニング
第13回	7/9（木）	深層学習の概観と使いどころ
第14回	7/16（木）	公平性・倫理と再現性
第15回	7/23（木）	まとめ

【Point】 やむなく欠席した場合、資料を確認のうえ不明ところは聞いてください。

1

分類とは

本日の学習内容を確認しよう



データマイニングによる分類の本質

>>> 分類 = 特徴空間に「境界」を引く学習（境界学習）

回帰： 出力は、連続値（いくらになるか？ いくつあるか？） 例：価格、気温、売上高

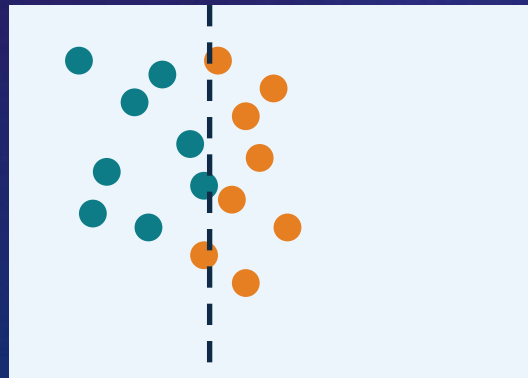
分類： 出力は、カテゴリ・ラベル（AかBか？、Yes or No？） 例：スパムか否か、犬か猫か

境界学習とは、特徴量の空間にデータを点として配置し、異なるクラスを分ける「境界線」をデータから自動で学ぶこと。

手法が違えば、境界の形も変わる。



kNN：曲線的な境界



決定木：縦横の直線的な境界

● クラスA ● クラスB - - - 決定境界

本日の環境準備 (I)

いろいろ試してみよう！

日本語対応のモジュールをインストールする(その1)

```
!pip install japanize-matplotlib
```

日本語対応のモジュールをインストールする(その2)

```
!apt-get install -y fonts-ipafont-gothic
```

本日の環境準備 (2)

いろいろ試してみよう！

各ライブラリーをインポートする

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
import japanize_matplotlib
import warnings
warnings.filterwarnings("ignore")

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import ( DecisionTreeClassifier, plot_tree)

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    classification_report,
    confusion_matrix,
    ConfusionMatrixDisplay
)

from sklearn.inspection import DecisionBoundaryDisplay
```

本日の環境準備 (3)

いろいろ試してみよう！

#データ読み込み

```
URL = ("https://data.insideairbnb.com/japan/kant%C5%8D/tokyo/2025-09-29/data/listings.csv.gz")
```

```
df = pd.read_csv(URL, compression="gzip", low_memory=False)
```

```
print(f"¥nデータサイズ:{df.shape[0]:,} 行 × {df.shape[1]} 列")
```

```
print("¥n主要列の確認")
```

```
print( df[[ "host_is_superhost",
            "price",
            "number_of_reviews",
            "review_scores_rating",
            "accommodates"
        ]].head())
```

```
print("¥nhost_is_superhost の分布")
```

```
print( df["host_is_superhost"].value_counts(dropna=False))
```

データサイズ : 27,945 行 × 79 列

主要列の確認

	host_is_superhost	price	number_of_reviews	review_scores_rating	¥
0	t	\$12,600.00	190	4.78	
1	t	\$10,459.00	272	4.98	
2	t	\$33,671.00	281	4.82	
3	t	\$24,143.00	284	4.95	
4	t	\$8,795.00	150	4.81	

accommodates

0	2
1	1
2	6
3	2
4	3

host_is_superhost の分布

host_is_superhost

f 14494

t 11435

NaN 2016

Name: count, dtype: int64

本日の環境準備（４）

前処理

ターゲット変数

```
df["is_superhost"] = (df["host_is_superhost"] == "t").astype(int)
```

price 前処理

通貨記号除去

```
df["price"] = (df["price"].astype(str).str.replace( r"^[0-9.]", "", regex=True))
```

数値変換

```
df["price"] = pd.to_numeric(df["price"], errors="coerce")
```

無限値除去

```
df["price"] = df["price"].replace([np.inf, -np.inf], np.nan)
```

欠損補完

```
price_median = df["price"].median()
```

```
print(f"¥nprice中央値:{price_median}")
```

```
df["price"] = df["price"].fillna(price_median)
```

負値防止

```
df["price"] = df["price"].clip(lower=0)
```

対数変換(外れ値対策)

```
df["price"] = np.log1p(df["price"])
```

#次に続く。。

本日の環境準備（5）

```
# minimum_nights 外れ値制限
```

```
df["minimum_nights"] = np.clip(df["minimum_nights"], 0, 30)
```

```
# 使用特徴量
```

```
# bedrooms は欠損や型崩れが多いため除外
```

```
FEATURES = ["price", "number_of_reviews", "review_scores_rating", "accommodates",  
            "minimum_nights", "availability_365"]
```

```
# 必要列抽出
```

```
df_clean = df[FEATURES + ["is_superhost"]].copy()
```

```
# 数値変換
```

```
for col in FEATURES:  
    df_clean[col] = pd.to_numeric(df_clean[col], errors="coerce")
```

```
# 欠損確認
```

```
print("\n【欠損数】")  
print(df_clean.isnull().sum())
```

```
# 欠損補完(中央値)
```

```
for col in FEATURES:  
    median_value = df_clean[col].median()  
    print(f"{col} の中央値: {median_value}")  
    df_clean[col] = df_clean[col].fillna(median_value)
```

```
#次に続く。。
```

本日の環境準備（6）

無限値除去

```
df_clean = df_clean.replace([np.inf, -np.inf], np.nan)
```

最終NaN除去

```
df_clean = df_clean.dropna()
```

確認

```
print("\n【補完後の欠損数】")
print(df_clean.isnull().sum())
print(f"\n前処理後:{df_clean.shape[0]:,} 行")
print(
    f"スーパーホスト比率:"
    f"{df_clean['is_superhost'].mean():.1%}"
)
```

特徴量・目的変数

```
X = df_clean[FEATURES]
y = df_clean["is_superhost"]
```

クラス分布

```
print("\n【クラス分布】")
print(y.value_counts())
print("\n割合")
print(y.value_counts(normalize=True))
```

#次に続く。。。

本日の環境準備 (7)

いろいろ試してみよう！

train / test 分割

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
print(f"¥n訓練データ:{len(X_train):,} 件")
print(f"テストデータ:{len(X_test):,} 件")
```

標準化(kNNで重要)

```
scaler = StandardScaler()
X_train_sc = scaler.fit_transform(X_train)
X_test_sc = scaler.transform(X_test)
```

NaN確認(超重要)

```
print("¥n【NaN確認】")
print("X_train_sc NaN数:", np.isnan(X_train_sc).sum())
print("X_test_sc NaN数 :", np.isnan(X_test_sc).sum())
```

NaNが残っていたら停止

```
if np.isnan(X_train_sc).sum() > 0:
    raise ValueError("X_train_sc に NaN が残っています")
```

price中央値: 9.692828247710418

【欠損数】

```
price          0
number_of_reviews  0
review_scores_rating  3936
accommodates    0
minimum_nights  0
availability_365  0
is_superhost    0
dtype: int64
price の中央値: 2.369573259521742
number_of_reviews の中央値: 16.0
review_scores_rating の中央値: 4.81
accommodates の中央値: 4.0
minimum_nights の中央値: 2.0
availability_365 の中央値: 159.0
```

【補完後の欠損数】

```
price          0
number_of_reviews  0
review_scores_rating  0
accommodates    0
minimum_nights  0
availability_365  0
```

```
is_superhost    0
dtype: int64
```

前処理後: 27,945 行
スーパーホスト比率: 40.9%

【クラス分布】

```
is_superhost
0    16510
1    11435
Name: count, dtype: int64
```

割合

```
is_superhost
0    0.590803
1    0.409197
Name: proportion, dtype: float64
```

訓練データ: 22,356 件
テストデータ: 5,589 件

【NaN確認】

```
X_train_sc NaN数: 0
X_test_sc NaN数: 0
```

本日の環境準備（8）

いろいろ試してみよう！

2次元可視化用データ

feat_x = "review_scores_rating"

feat_y = "number_of_reviews"

X2 = df_clean[[feat_x, feat_y]].values

y2 = df_clean["is_superhost"].values

```
X2_train, X2_test, y2_train, y2_test = train_test_split(
    X2,
    y2,
    test_size=0.2,
    random_state=42,
    stratify=y2
)
```

scaler2 = StandardScaler()

X2_train_sc = scaler2.fit_transform(X2_train)

X2_test_sc = scaler2.transform(X2_test)

2

k近傍法 (kNN)

近くにあるものと同じクラス



k近傍法 (kNN) とは (1)

>>> 人間が自然にやっている分類をそのまま数式にしたシンプルな手法

- ① 未知のデータと全訓練データとの距離を計算
- ② 距離が近い順にk個の近傍を選ぶ
- ③ k個の多数決でクラスを決定

<kの選び方>

- | | | | |
|-------------|---|---------------------|-----------------|
| k = 1 (小さい) | : | 複雑でこぼこした境界 | ⇒ 過学習 (新データに弱い) |
| k が大きい | : | 境界がのっぺり | ⇒ 未学習 (精度が低い) |
| k が最適 | : | 適度に滑らかな境界 (バランスが良い) | |

kNNは距離を使う手法のため、必ず `StandardScaler()` で標準化してから学習・予測すること。
スケールが異なる変数が混在すると、値が大きい変数だけが距離を支配してしまう。

<kNNの弱点>

- | | | |
|--------|---|---|
| 予測が遅い | : | 予測ごとに全訓練データとの距離を計算する必要がある。データが大きいほど遅い。 |
| 次元の呪い | : | 高次元 (変数が多い) になるほど、すべての点が「遠く」なり距離が意味を失う。 |
| 解釈が難しい | : | 「なぜこの分類結果になったか」を人間が読める形で説明できない。
⇒ やはり、決定木の良さそうか。。。 |

k近傍法 (kNN) とは (2)

```
# kNN
k_values = range(1, 26, 2)
train_scores = []
test_scores = []
f1_scores = []
for k in k_values:
    knn = KNeighborsClassifier(n_neighbors=k, weights="distance")
    knn.fit(X_train_sc, y_train)
    y_train_pred = knn.predict(X_train_sc)
    y_test_pred = knn.predict(X_test_sc)
    train_acc = accuracy_score(y_train, y_train_pred)
    test_acc = accuracy_score(y_test, y_test_pred)
    test_f1 = f1_score(y_test, y_test_pred)
    train_scores.append(train_acc)
    test_scores.append(test_acc)
    f1_scores.append(test_f1)
```

次に行く。

k近傍法 (kNN) とは (3)

いろいろ試してみよう！

```
# 最適k
best_idx = np.argmax(f1_scores)
best_k = list(k_values)[best_idx]
gap = (train_scores[best_idx] - test_scores[best_idx])
print(f"※最適な k:{best_k}")
print(
    f"訓練Accuracy:"
    f"{train_scores[best_idx]:.3f}"
)
print(
    f"テストAccuracy:"
    f"{test_scores[best_idx]:.3f}"
)
print(
    f"F1-score:"
    f"{f1_scores[best_idx]:.3f}"
)

print(
    f"過学習ギャップ:"
    f"{gap:.3f}"
)

# 次に行く。
```

最適な k : 15
訓練Accuracy : 0.998
テストAccuracy : 0.732
F1-score : 0.654
過学習ギャップ : 0.266

k近傍法 (kNN) とは (4)

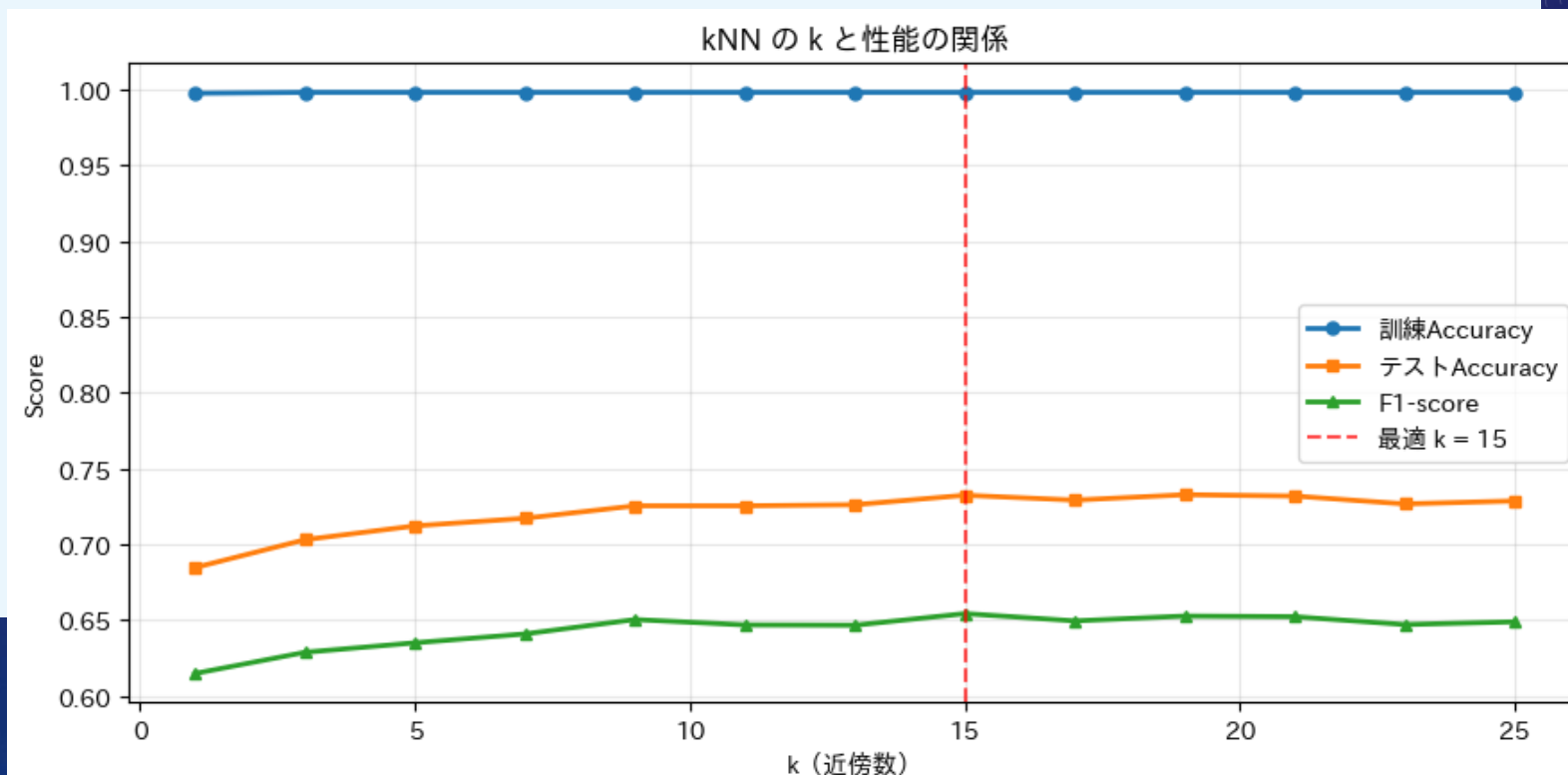
いろいろ試してみよう！

可視化

```
fig, ax = plt.subplots(figsize=(9, 4.5))
ax.plot(list(k_values), train_scores, "o-", linewidth=2, markersize=5, label="訓練Accuracy")
ax.plot(list(k_values), test_scores, "s-", linewidth=2, markersize=5, label="テストAccuracy")
ax.plot(list(k_values), f1_scores, "^-", linewidth=2, markersize=5, label="F1-score")
```

```
ax.axvline(
    x=best_k,
    color="red",
    linestyle="--",
    alpha=0.7,
    label=f"最適 k = {best_k}"
)
```

```
ax.set_xlabel("k(近傍数)")
ax.set_ylabel("Score")
ax.set_title("kNN の k と性能の関係")
ax.grid(True, alpha=0.3)
ax.legend()
plt.tight_layout()
plt.show()
```



k近傍法 (kNN) とは (5)

いろいろ試してみよう！

kNN 決定境界

```
fig, axes = plt.subplots(1, 3, figsize=(15, 4.5))
k_list = [1, best_k, 21]
titles = ["k=1(過学習)", f"k={best_k}(最適)", "k=21(単純化)"]
for ax, k, title in zip(axes, k_list, titles):
    knn2 = KNeighborsClassifier(n_neighbors=k)
    knn2.fit(X2_train_sc, y2_train)

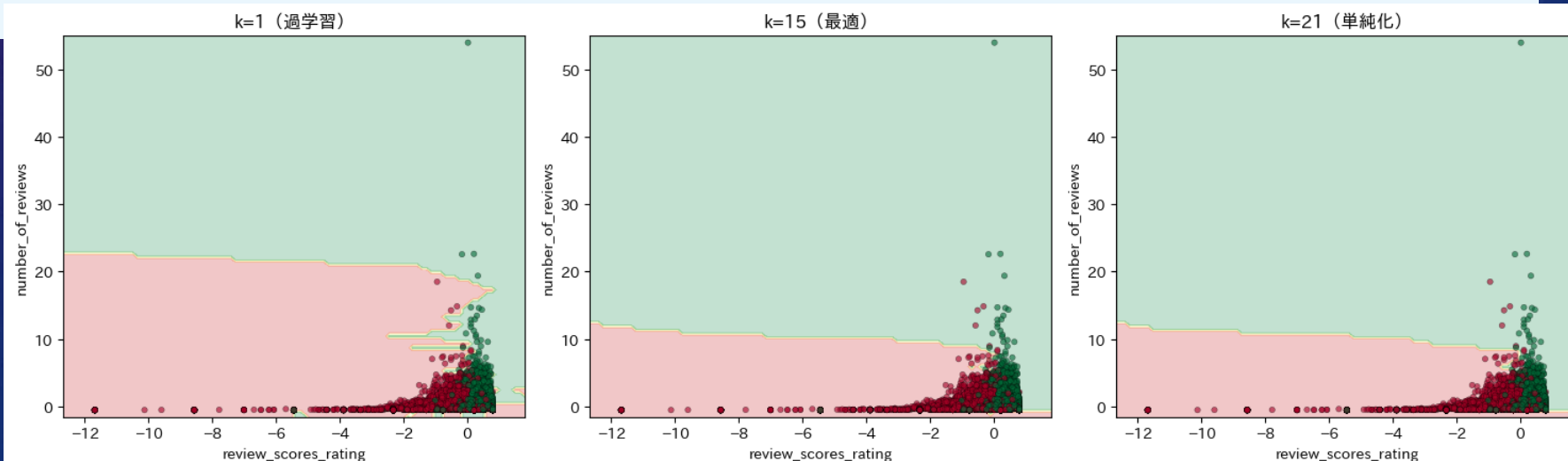
    DecisionBoundaryDisplay.from_estimator(
        knn2,
        X2_train_sc,
        response_method="predict",
        alpha=0.25,
        cmap="RdYlGn",
        ax=ax
    )
```

次へ続く。

k近傍法 (kNN) とは (6)

いろいろ試してみよう！

```
ax.scatter(  
    X2_train_sc[:, 0],  
    X2_train_sc[:, 1],  
    c=y2_train,  
    cmap="RdYlGn",  
    edgecolors="k",  
    linewidths=0.3,  
    s=15,  
    alpha=0.6  
)  
  
ax.set_title(title)  
ax.set_xlabel(feats_x)  
ax.set_ylabel(feats_y)  
plt.tight_layout()  
plt.show()
```



3

決定木

Yes/Noで分岐するフローチャート



The background is a dark blue gradient with a subtle pattern of white dots. Overlaid on this are several faint, light blue circular elements. On the left side, there is a large circular scale with tick marks and numbers ranging from 140 to 260. Several smaller circles, some with arrows indicating a clockwise direction, are scattered across the upper and lower portions of the image.

決定木

決定木 (1)

>>> 人間が自然にやっている判断を、データから自動的に構築する手法

<どこで分割するか — 情報利得とジニ不純度>

基本的な考え方：「分割後の各グループがなるべく純粋（1種類のみ）になる
分割を選ぶ」

完全に不純（最悪）	：	クラスAとBが半々	⇒ジニ不純度 = 0.5
混ざっている	：	クラスAが70%、Bが30%	⇒ジニ不純度 = 0.42
完全に純粋（最良）	：	クラスAのみ100%	⇒ジニ不純度 = 0

評価スコア ≥ 4.8 ?

Yes

スーパーホスト

No

レビュー件数 ≥ 20 ?

決定木 (2)

いろいろ試してみよう！

決定木

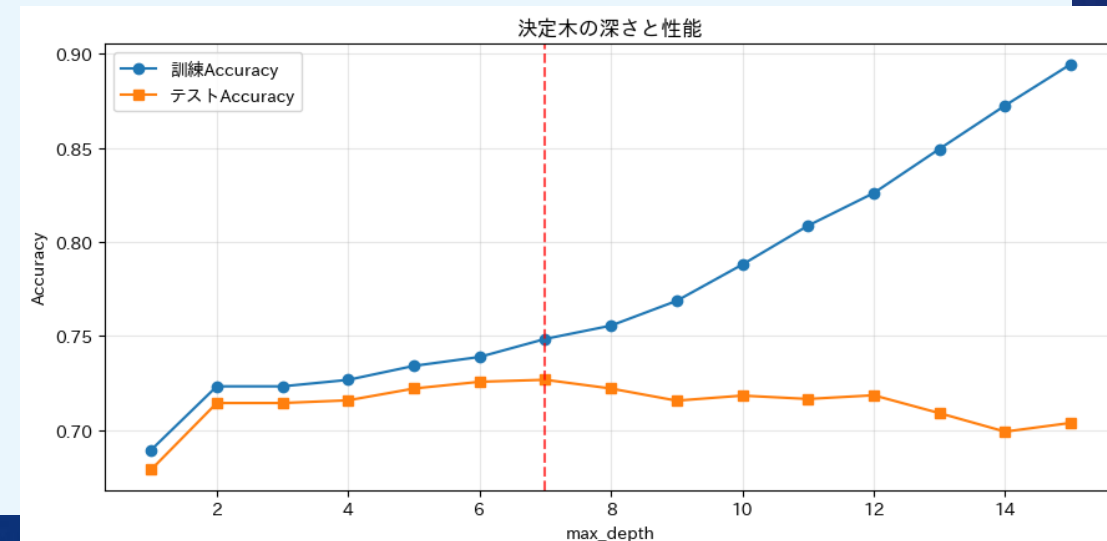
```
depths = range(1, 16)
dt_train = []
dt_test = []
for d in depths:
    dt = DecisionTreeClassifier(max_depth=d, random_state=42)
    dt.fit(X_train, y_train)
    dt_train.append(dt.score(X_train, y_train))
    dt_test.append(dt.score(X_test, y_test))

best_depth = (np.argmax(dt_test) + 1)
print(f"※最適深さ:{best_depth}")

fig, ax = plt.subplots(figsize=(9, 4.5))
ax.plot(depths, dt_train, "o-", label="訓練Accuracy")

ax.plot(depths, dt_test, "s-", label="テストAccuracy")
ax.axvline(x=best_depth, color="red", linestyle="--", alpha=0.7)
ax.set_xlabel("max_depth")
ax.set_ylabel("Accuracy")
ax.set_title("決定木の深さと性能")
ax.grid(True, alpha=0.3)

ax.legend()
plt.tight_layout()
plt.show()
```



決定木 (3)

いろいろ試してみよう！

決定木可視化

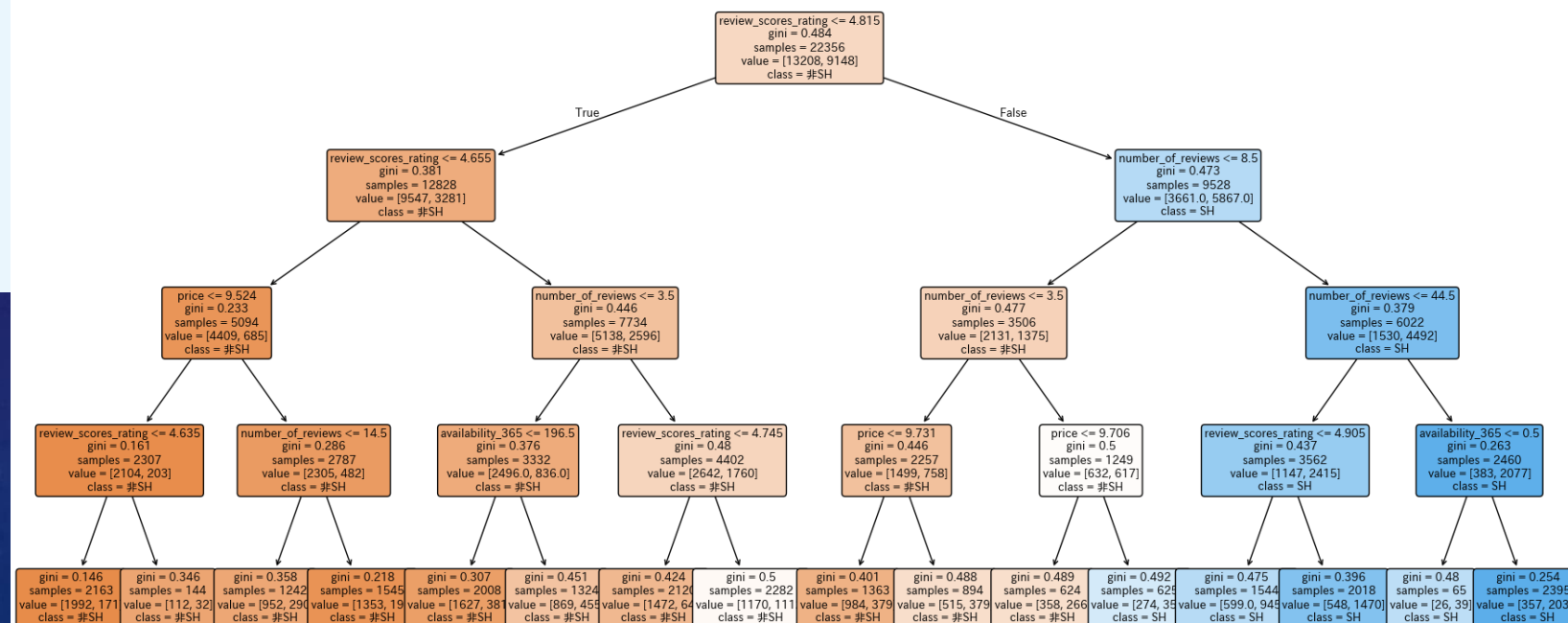
```
dt_final = DecisionTreeClassifier(max_depth=4, random_state=42)
```

```
dt_final.fit(X_train, y_train)
```

```
fig, ax = plt.subplots(figsize=(18, 8))
```

```
plot_tree(
    dt_final,
    feature_names=FEATURES,
    class_names=["非SH", "SH"],
    filled=True,
    rounded=True,
    fontsize=9,
    ax=ax
)
```

```
plt.tight_layout()
plt.show()
```



The background is a dark blue gradient with a subtle pattern of white dots. Overlaid on the left side are several concentric circular patterns. A prominent circular scale with degree markings from 140 to 260 is visible. Other circles with arrows indicating clockwise or counter-clockwise rotation are scattered across the left half of the image. The text '特微量重要度' is centered on the right side, featuring a yellow-to-white gradient and a soft glow effect.

特微量重要度

特徴量重要度（I）

>>> 特徴量重要度 — 人間が読めるAIの強み

決定木は「人間が目で読めるモデル」の代表格。
なぜその判断になったかをフローチャートで示せる点が他の多くの手法にはない強み。

（※ LIMEやSHAPなど後発の説明可能AI手法もあるが、構造の直感的な理解しやすさでは決定木が群を抜く）

Airbnbデータで期待される結果

- 1 位：review_scores_rating（評価スコアが最重要）
- 2 位：number_of_reviews（実績の多さ）
- 3 位：price（価格）

特徴量重要度 (2)

いろいろ試してみよう！

特徴量重要度

```
importance_df = pd.DataFrame({"特徴量": FEATURES, "重要度": dt_final.feature_importances_})  
importance_df = importance_df.sort_values("重要度", ascending=True)
```

```
fig, ax = plt.subplots(figsize=(8, 4.5))
```

```
ax.barh(importance_df["特徴量"], importance_df["重要度"])
```

```
ax.set_xlabel("重要度")
```

```
ax.set_title("特徴量重要度")
```

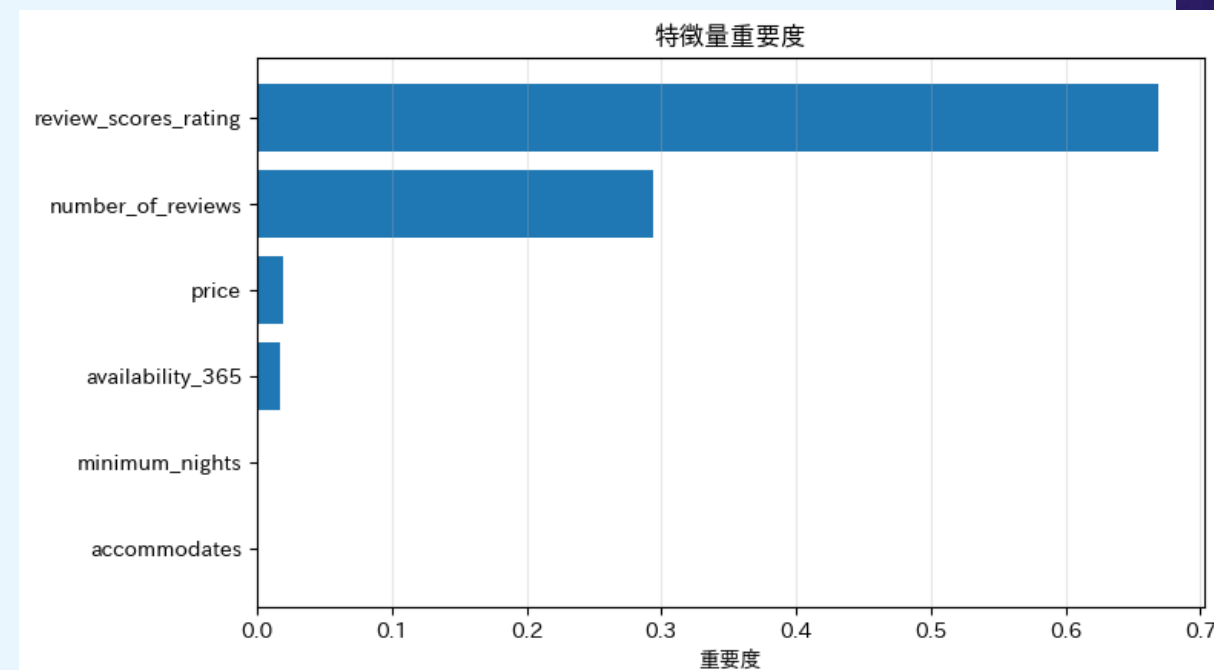
```
ax.grid(True, axis="x", alpha=0.3)
```

```
plt.tight_layout()
```

```
plt.show()
```

```
print("\n重要度(降順)")
```

```
print(importance_df.sort_values("重要度", ascending=False))
```



重要度 (降順)

	特徴量	重要度
2	review_scores_rating	0.669056
1	number_of_reviews	0.293693
0	price	0.019923
5	availability_365	0.017327
4	minimum_nights	0.000000
3	accommodates	0.000000

The background is a deep blue gradient with a subtle pattern of white dots. Overlaid on this are several faint, white geometric elements: concentric circles, arcs, and degree markings. A large arc on the left side is marked with degrees from 140 to 260 in increments of 10. Other smaller arcs and circles are scattered across the frame, some with arrows indicating a clockwise direction.

混同行列

混同行列（I）

>>> 「正解率」だけでは見えないものがある

仮に「全員を非スーパーホストと予測するだけ」のモデルでも、正解率75%が出てしまう

<混同行列が教えてくれること>

1. 精度 : 全体の正解率。不均衡データでは意味が薄い
2. 再現率 : 本物のSHを何%検出できたか
3. 適合率 : SHと予測した中で本当にSHの割合
4. F1スコア : 再現率と適合率の調和平均

※**Airbnbのスーパーホスト（SH）**は、
高い評価と優れたホスピタリティを
持つホストに与えられる公式称号

混同行列（Confusion Matrix）

	予測：非SH	予測：SH
実際：非SH	TN 真の非SH	FP 非SHをSHと誤判定
実際：SH	FN SHを見逃した	TP 真のSH

混同行列（2）

最終評価

kNN

```
knn_final = KNeighborsClassifier(n_neighbors=best_k, weights="distance")
knn_final.fit(X_train_sc, y_train)
y_pred_knn = knn_final.predict(X_test_sc)
```

決定木

```
y_pred_dt = dt_final.predict(X_test)
```

レポート

```
for name, y_pred in [("kNN", y_pred_knn), ("決定木", y_pred_dt)]:
    print("\n" + "=" * 40)
    print(name)
    print("=" * 40)
    print(classification_report(y_test, y_pred, target_names=["非SH", "SH"], digits=3))
```

混同行列

```
fig, axes = plt.subplots(1, 2, figsize=(11, 4.5))
for ax, (name, y_pred) in zip(axes, [("kNN", y_pred_knn), ("決定木", y_pred_dt)]):
    cm = confusion_matrix(y_test, y_pred)
    disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=["非SH", "SH"])
    disp.plot(ax=ax, colorbar=False, cmap="Blues")
    ax.set_title(name)
```

```
plt.tight_layout()
plt.show()
```

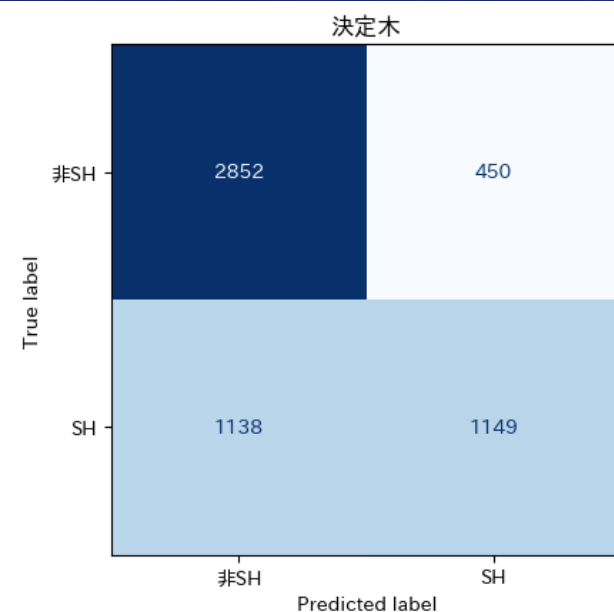
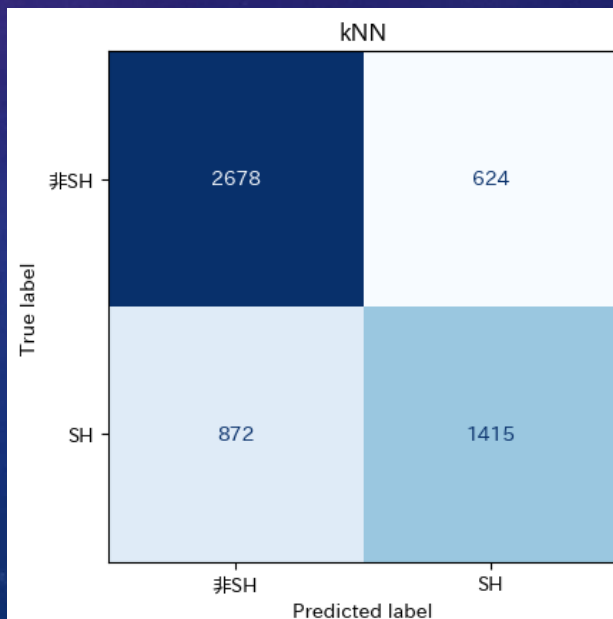
混同行列 (3)

kNN

	precision	recall	f1-score	support
非SH	0.754	0.811	0.782	3302
SH	0.694	0.619	0.654	2287
accuracy			0.732	5589
macro avg	0.724	0.715	0.718	5589
weighted avg	0.730	0.732	0.730	5589

決定木

	precision	recall	f1-score	support
非SH	0.715	0.864	0.782	3302
SH	0.719	0.502	0.591	2287
accuracy			0.716	5589
macro avg	0.717	0.683	0.687	5589
weighted avg	0.716	0.716	0.704	5589



The background is a gradient from dark purple at the top to dark blue at the bottom, speckled with small white dots. On the left side, there are several concentric circular patterns. One large circle has a degree scale from 140 to 260. Other smaller circles have arrows indicating clockwise or counter-clockwise rotation. The text 'まとめ' is centered in the middle of the image.

まとめ

まとめ

観点	K近傍法(kNN)	決定木
境界の形	曲線的・複雑	縦横の直線的な分割
解釈性	× 難しい	◎ 目で読めるモデルの代表格
速度(予測)	× 遅い(全データと比較)	◎ 高速(木を辿るだけ)
高次元への対応	△ 次元の呪いに弱い	○ 比較的得意
スケーリング	★ 必須	不要
過学習の制御	k の値で調整	max_depth で制御
主な用途	プロトタイピング・直感確認	説明責任が必要な場面

4

本日の課題

頑張ってください。



課題

【課題】 期限：2026/5/20（水） 23：59まで

WebClassの『データマイニング特論』アクセスし、
本日の日付が記載される「課題」を期限内に実施してください。

・ WebClass -> データマイニング特論 -> 【第5回（5/14）】課題

（課題1） 本日の演習で「いろいろ試してみよう！」の中で実施した入出力コードを
html形式でエクスポートし、提出しなさい。

html形式にする手順

- ①ipynb形式でダウンロード（ファイル名の例：aaa.ipynb）
- ②再び「ファイル」へアップロード
- ③以下のコマンドを入力

```
!jupyter nbconvert --to html "aaa.ipynb"
```